# DEVOPS WINTER ARC

90 Days Challenge • Kubernetes Week 3

# DAY 18

ConfigMaps, Secrets & Namespaces

| | |
|---|---|
| **Learning focus** | Environment-specific configuration, Secret references, namespace isolation, and mounted configuration files. |
| **Expected outcome** | Run the same Nginx workload in **dev** and **prod** with different page content and configuration. |
| **Submission** | GitHub folder, YAML files, evidence screenshots, and short answers. |

## 1. Scope rule — read before starting

**Do not repeat earlier assignments.** Day 15 covered Kubernetes fundamentals, initial Nginx Deployment/Service setup, scaling, and NodePort/port-forward access. Day 16 covered rolling updates, revision history, failed image updates, and rollback. Day 17 covered ClusterIP service discovery, DNS, EndpointSlices, traffic distribution, and selector troubleshooting. **Do not add those tasks again.** Day 18 is limited to ConfigMaps, Secrets, namespaces, environment variables, and mounted configuration files.

## 2. Learning objectives

- Create **dev** and **prod** namespaces.
- Store non-sensitive application settings in a ConfigMap.
- Reference a Secret from a container environment variable without putting credentials in an image or Deployment manifest.
- Mount ConfigMap content as files and compare environment-specific output.
- Explain how mounted files and environment variables behave when configuration changes.

## 3. Task A — Create isolated namespaces

Create the two namespaces and verify them:

```
kubectl create namespace dev
kubectl create namespace prod
kubectl get namespaces
```

If a namespace already exists, inspect it instead of deleting it.

## 4. Task B — Create different ConfigMaps

Save the following as **configmap-dev.yaml**. Create **configmap-prod.yaml** similarly, but use **APP_ENV: prod** and **Hello from PROD** in the page.

```
apiVersion: v1
kind: ConfigMap
metadata:
name: site
data:
APP_ENV: dev
index.html: |
<h1>Hello from DEV</h1>
<p>Winter Arc · Day 4</p>
```

```
kubectl apply -f configmap-dev.yaml -n dev
kubectl apply -f configmap-prod.yaml -n prod
kubectl get configmap site -n dev
kubectl get configmap site -n prod
```

## 5. Task C — Create lab-only Secrets

**Use dummy values only.** Never commit real passwords, tokens, or production secrets to GitHub.

```
kubectl create secret generic db-secret \
--from-literal=password=dev-pass -n dev
kubectl create secret generic db-secret \
--from-literal=password=prod-pass -n prod
kubectl get secret db-secret -n dev
kubectl get secret db-secret -n prod
```

## 6. Task D — Configure the same workload in both namespaces

Save as **deployment.yaml**. The ConfigMap provides APP_ENV and the Nginx page; the Secret provides DB_PASSWORD as an environment variable.

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: web
spec:
replicas: 1
selector:
matchLabels:
app: web
template:
metadata:
labels:
app: web
spec:
containers:
- name: nginx
image: nginx:1.27
ports:
- containerPort: 80
env:
- name: APP_ENV
valueFrom:
configMapKeyRef:
name: site
key: APP_ENV
- name: DB_PASSWORD
valueFrom:
secretKeyRef:
name: db-secret
key: password
volumeMounts:
- name: html
mountPath: /usr/share/nginx/html
volumes:
- name: html
configMap:
name: site
```

```
kubectl apply -f deployment.yaml -n dev
kubectl apply -f deployment.yaml -n prod
kubectl get pods -n dev
kubectl get pods -n prod
```

## 7. Task E — Verify each environment

Run each port-forward command in a separate terminal and keep it running:

```
kubectl port-forward -n dev deploy/web 8081:80
kubectl port-forward -n prod deploy/web 8082:80
```

From another terminal, check both pages and environment variables:

```
curl http://localhost:8081
curl http://localhost:8082
kubectl exec -n dev deploy/web -- printenv APP_ENV
kubectl exec -n dev deploy/web -- printenv DB_PASSWORD
kubectl exec -n prod deploy/web -- printenv APP_ENV
kubectl exec -n prod deploy/web -- printenv DB_PASSWORD
```

Expected: the dev page says **Hello from DEV** and APP_ENV=dev; prod says **Hello from PROD** and APP_ENV=prod. Do not show DB_PASSWORD values in public screenshots or your README.

## 8. Task F — Update configuration and observe

Edit the dev ConfigMap and change its page text and APP_ENV:

```
kubectl edit configmap site -n dev
```

Check the page again after a short delay. Mounted ConfigMap files usually update after a short delay (often around a minute), but environment variables are set when the container starts. Restart the Deployment to load a changed APP_ENV:

```
kubectl rollout restart deployment/web -n dev
kubectl rollout status deployment/web -n dev
kubectl exec -n dev deploy/web -- printenv APP_ENV
```

Note: ConfigMap volumes mounted with **subPath** do not receive automatic updates. This assignment mounts the ConfigMap as a directory.

## 9. Troubleshooting checklist

| Symptom | What to check |
|---|---|
| CreateContainerConfigError | Check that the ConfigMap/Secret exists in the same namespace and the key names match. |
| Resources appear missing | Check namespaces with kubectl get all -n dev and kubectl get all -n prod. |
| Wrong page appears | Inspect ConfigMap data and the /usr/share/nginx/html mount path. |
| APP_ENV did not change | Restart the Deployment and inspect the new Pod's environment. |

## 10. Security rules

- Kubernetes Secret data is base64-encoded by default; base64 is **not encryption**.
- Use dummy credentials and never push real secrets to GitHub.
- Restrict Secret access with RBAC. Production systems may require encryption at rest and a dedicated secrets manager.

## 11. Submission checklist

Create a GitHub folder named **day18-configmaps-secrets-namespaces/** containing:

- ■ configmap-dev.yaml and configmap-prod.yaml
- ■ deployment.yaml
- ■ README.md with setup steps and observations
- ■ Screenshots of both pages, namespace-specific environment values (mask passwords), and config-update behavior
- ■ Answers to the questions below

## 12. Interview / reflection questions

1. What is the difference between a ConfigMap and a Secret?
2. Why must referenced ConfigMaps and Secrets exist in the Pod's namespace?
3. What is the difference between environment variables and mounted files?
4. Why does changing a ConfigMap not update an existing container's environment automatically?
5. Why is base64 not sufficient to protect a Secret?
6. What can cause CreateContainerConfigError?
7. How do namespaces help separate dev and prod resources?

## 13. Cleanup — after saving evidence

```
kubectl delete namespace dev
kubectl delete namespace prod
```

This removes all lab resources in those namespaces. Run cleanup only after collecting your evidence.

**Completion standard:** Both namespaces run the same workload image but show different content; ConfigMap and Secret references work; and you can explain mounted-file versus environment-variable update behavior.